

CASE STUDY

Laravel Ordering System

A role-based web application for catalog browsing, cart checkout, order processing, and inventory control

"A beginner-friendly Laravel system that turns manual order handling into a structured e-commerce workflow."

Project Type	Student web application / ordering system
Technology Stack	Laravel, PHP, MySQL, Blade, Bootstrap 5, jQuery DataTables, SweetAlert2
Primary Users	Admin, Staff, Customer
Prepared Date	August 13, 2026

Case Study Snapshot

The application demonstrates a complete small-business ordering workflow: customers browse active items, place orders through a cart, staff process orders, and administrators manage users, inventory, and records.

Executive Summary

The Laravel Ordering System is a complete web-based ordering application designed for a small food, retail, or student demonstration environment. It replaces informal order tracking with a structured workflow that supports user roles, item management, cart-based ordering, stock control, and order status processing.

The project emphasizes standard Laravel conventions and beginner-friendly implementation. It uses Blade templates instead of a single-page frontend framework, Eloquent relationships for clean data access, Form Requests for validation, middleware for role authorization, and database transactions for order and inventory consistency.

Project Background

Small ordering workflows often begin with manual lists, chat messages, spreadsheets, or paper forms. These methods are easy to start but become unreliable once multiple users, item availability, stock quantities, and order status updates are involved.

This application was built to show how a conventional Laravel project can organize those workflows into a maintainable web system. The result feels like a lightweight e-commerce platform while still keeping the codebase understandable for students and early Laravel developers.

Objectives

- Provide a public home page that displays all active items available for purchase.
- Allow customers to register, log in, browse the catalog, manage a cart, submit orders, and review their own order history.
- Allow staff to view and process orders without granting administrator privileges.
- Allow administrators to manage staff, customers, items, orders, and system records.
- Deduct stock when an order is submitted and restore stock when an eligible order is cancelled.
- Use server-side validation and authorization to protect prices, totals, stock, and customer ownership.

User Roles and Capabilities

Role	Primary Responsibilities	Key Restrictions
Admin	Manages users, staff, customers, items, orders, account status, item status, and dashboard statistics.	Must still follow server-side order status rules and inventory constraints.
Staff	Processes customer orders, views available items, updates order status, and creates orders for customers when needed.	Cannot manage administrator accounts or access admin-only user management features.
Customer	Browses items, adds items to cart, submits orders, views own orders, and cancels own pending orders.	Cannot view or manipulate orders that belong to other customers.

Solution Overview

The application combines an e-commerce-style storefront with administrative order management. Guests can view the shop home page, while authenticated customers can purchase active in-stock items. Staff and administrators work through role-specific dashboards and shared order-management screens.

A reusable Blade layout provides a top navigation bar, role-aware sidebar, flash messages rendered through SweetAlert2, DataTables for searchable administrative tables, and responsive Bootstrap components for desktop and mobile use.

Core Features

- Public storefront home page with product cards, prices in Philippine pesos, item stock indicators, and default product avatars.
- Customer catalog and session-based cart with quantity updates, remove, clear, total review, and submit order confirmation.
- Admin item management with add, edit, view, delete, activate, and deactivate actions.
- Admin staff and customer management with account activation controls.
- Order listing, status filtering, order details, order item breakdown, and role-specific visibility.
- SweetAlert2 confirmations for destructive and important actions such as delete, cancel, and submit order.
- Email-based local/demo password update flow for simple account recovery during development.

Technical Architecture

Layer	Implementation	Purpose
Routing	Laravel named routes grouped by auth and role middleware	Keeps URLs organized by Admin, Staff, and Customer workflows.
Controllers	Dashboard, Auth, Staff, Customer, Item, Cart, and Order controllers	Separates user-facing workflows from business operations.
Models	User, Item, Order, and OrderItem Eloquent models	Defines relationships and protects data access through Laravel conventions.
Views	Blade templates with Bootstrap 5 and partials	Provides reusable UI patterns without a SPA framework.
Database	MySQL-ready migrations with foreign keys and decimal money fields	Stores users, items, orders, and order line items reliably.
Frontend Libraries	Bootstrap Icons, jQuery DataTables, SweetAlert2	Improves usability with icons, searchable tables, and confirmations.

Database Design

The database centers on four domain tables: users, items, orders, and order_items. Users carry role and status fields. Items store current catalog data, stock quantity, status, and price. Orders record the customer, creator, total amount, status, notes, and human-readable order number. Order items preserve the unit price at the time the order was created.

Table	Important Fields	Relationship Role
users	name, email, password, role, status	Customers own orders; staff/admin users may create orders.
items	name, description, price, stock_quantity, status	Items are selected into order item lines.
orders	order_number, customer_id, total_amount, status, notes, created_by	Parent record for order history and processing.
order_items	order_id, item_id, quantity, unit_price, subtotal	Stores line-level quantity, price snapshot, and subtotal.

Order and Inventory Workflow

Order creation is intentionally handled server-side. The browser can submit item IDs and quantities, but it cannot control prices, stock values, subtotals, or grand totals. During checkout, the system retrieves current items from the database, locks the rows, validates stock and item status, computes subtotals, creates the order, creates order item records, deducts inventory, and commits the transaction.

Step	System Action	Reason
1	Validate the cart or order creation form.	Rejects missing, invalid, or zero-quantity lines before processing.
2	Open a database transaction and lock selected item rows.	Prevents overselling when multiple users order at the same time.
3	Read price and stock directly from the database.	Prevents browser-side price or total manipulation.
4	Create the order and order_items records.	Stores a durable order history and line-level price snapshot.
5	Deduct stock after successful creation.	Reserves inventory immediately once an order is submitted.
6	Restore stock when an eligible order is cancelled.	Keeps inventory accurate and makes cancellation idempotent.

Status Workflow

The normal order status flow is: Pending -> Confirmed -> Processing -> Completed.

- Completed orders cannot be changed back to pending.
- Cancelled orders cannot be processed further.
- Customers can cancel only their own pending orders.
- Admin and staff can cancel eligible pending, confirmed, or processing orders through the cancellation transaction.
- Selecting cancelled from an admin or staff status update is routed through the same stock-restoring cancellation logic.

Security and Authorization

The application uses Laravel authentication, password hashing, CSRF protection, role middleware, route grouping, validation, and explicit ownership checks. Customer order access is protected against IDOR by verifying that the requested order belongs to the authenticated customer before rendering details.

Risk	Control Implemented	Result
Unauthorized role access	Role middleware on Admin, Staff, and Customer route groups	Users can only reach permitted sections.
Inactive account access	Middleware logs out inactive users and returns them to login	Disabled accounts cannot continue using the system.
Price or total tampering	Server-side price retrieval and subtotal calculation	Browser-submitted prices are ignored.
Overselling stock	Database transactions and row locking on selected items	Concurrent checkout risk is reduced.
Customer data leakage	Customer ownership checks on order details	Customers cannot view other customers' orders.

User Interface Design

The interface uses Bootstrap 5 for a clean, responsive experience. The public home page presents the application as a store, while authenticated users see dashboards and role-aware navigation. DataTables improve management screens with search, sorting, pagination, and page-length controls. SweetAlert2 gives create, update, delete, cancel, and submit actions clearer feedback and confirmation.

Every item now displays a default product avatar, which gives the storefront and management screens a more complete e-commerce feel even before real product images are uploaded.

Testing and Verification

The project includes feature tests for role-based login redirects, customer checkout with stock deduction, customer order ownership protection, and email-based password update. The implementation was also checked with PHP syntax validation, route registration, Blade compilation, migration/seeding, and local HTTP smoke checks.

Verification Area	Check Performed	Observed Outcome
Authentication	Feature test for login redirect by role	Passing
Checkout	Feature test for cart submission and stock deduction	Passing
Authorization	Feature test preventing another customer from viewing an order	Passing
Password Update	Feature test for updating password by registered email	Passing
Rendering	Blade templates compiled successfully	Passing

Limitations

- The current password update flow is email-based and intended for local/demo use; production recovery should use signed reset tokens or email verification.
- Item images currently use a default avatar rather than per-item uploaded images.
- The system does not yet include payments, delivery tracking, refunds, taxes, or receipt generation.
- Reporting is limited to dashboard counters and order tables.
- The UI uses CDN assets, so offline production deployments should bundle frontend dependencies locally.

Recommended Enhancements

Enhancement	Value	Priority
Per-item image upload	Improves storefront realism and product recognition.	High
Token-based password reset	Makes account recovery production-ready.	High
Order receipts or invoices	Provides printable transaction records.	Medium
Payment integration	Completes the e-commerce purchase cycle.	Medium
Sales and inventory reports	Helps administrators make operational decisions.	Medium

Enhancement	Value	Priority
Category and search filters	Improves catalog browsing as item count grows.	Low

Conclusion

The Laravel Ordering System successfully demonstrates how a small ordering process can become a structured e-commerce-style application. It balances usability and maintainability by using familiar Laravel patterns, server-side business logic, responsive Blade views, and practical security checks.

As a student project, it is complete enough to demonstrate real workflows while remaining approachable for explanation, presentation, and future extension. With image uploads, production password recovery, payment handling, and reporting, the system could evolve into a stronger small-business ordering platform.